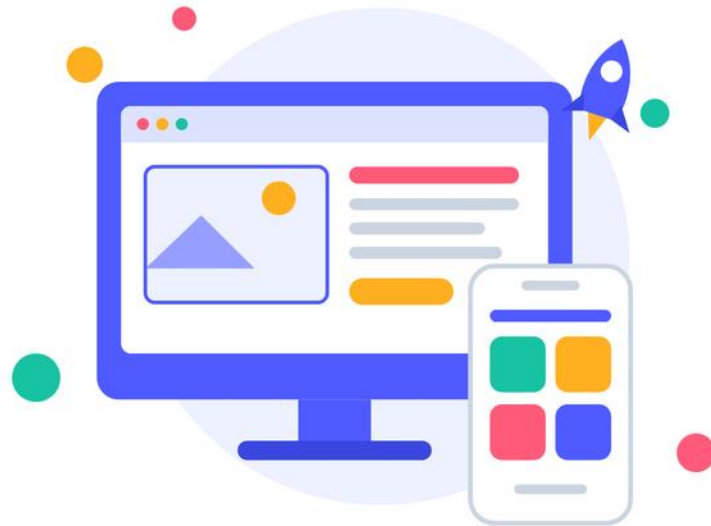


WEEK 02 · HTML II · FORMS, TABLES, AND THE READER WHO CANNOT SEE

THE READER WHO CANNOT SEE

Last week: a page a browser can parse. Today: one a person can use without a mouse or eyes.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Access by everyone regardless of disability is an essential aspect.

TIM BERNERS-LEE

WHERE WE ARE

Week 2 of 12. Your week-1 page is in your repo; today it grows a form, a table and a way in.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: a form · a table · the reader · one review from Claude · details and picture. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

Next week the CSS goes on exactly this markup. Every label and landmark you type today is a selector then.

MINUTE 2 · THE PROBLEM

YOUR FORM LOOKS FINISHED. IT IS EMPTY.

Grey words above each box, a hint inside it, a Send button. It validates with zero errors. Three things are wrong with it, and none of them is visible.

1. Click the words. **Nothing happens** — they are not attached to anything.
2. Press Send. The address bar shows a bare `?` — **no field has a name**, so nothing was sent.
3. Close your eyes. The hint vanished the moment you typed, and the reader hears a box with a name it borrowed from the placeholder.

THIS WEEK'S RULE

Green is a floor, not the standard. The validator passed this form. An accessibility report will pass it too (cycle 3, and again in cycle 4). The standard is what a person can do: click, type, hear, Tab.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment — week 1's page, grown; still no CSS, on purpose

↶ reload

open ↗

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 — from 700px

- [Home](#)
- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 — from 700px

- [Home](#)
- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me



targets/week-02.png · 1280

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

375 · the narrow banner, nothing overflows

► **THE POINT** — same page as last week, plus a form every word of which is clickable, a table a screen reader can walk, a date a machine can read, a skip link, a nav that says where you are, and a banner that picks its own source. You type every piece of it today: once in a cycle, once in the assignment.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1-5 · PRIVATE TUTOR

ALLOWED

Ask it what `scope` does, why a `for` must match an `id`, what a Lighthouse row means. Ask it to review a form *you* typed. Then check the answer in the browser — click the words, press Tab, open the Accessibility pane — not in your gut.

NOT ALLOWED

Pasting generated markup into the assignment. **You type every character.** The tell this week: "add `aria-label`, axe will pass". It does pass. Cycle 4 shows you what it did not check.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one review, twice, in whatever chat you have.

CYCLE 1 · 25 MIN

THE FORM THAT FAILS WELL

The film club's sign-up form: grey words, placeholders, no names. Click, Send, validate — three tools, three silences. Then one attribute at a time until every word is a handle and every field has a name.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/01-form-that-fails-well · label for + id + name · type + required · fieldset + legend · aria-describedby · a radio group

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 1 · every label focuses its field

3 YOU DO

8 MIN

warmup/ — the booking form from target.png · two input types you have to guess

4 DEBRIEF

2 MIN

placeholder is not a label · id vs name · ↗ reference

YOU DO · AI-FREE

CYCLE 1 · BEFORE YOU TYPE

CLICK THE WORDS "FULL NAME"

demos/05-forms/01-broken — nothing happens

↻ reload

open ↗

Contact us

Full name

Email

Message

Press "send" to see what the server would receive.

► **HANDS-ON** demos/05-forms/ — Click the words "Full name". Nothing. Press send . Nothing. Then run the validator on it — it passes. And in cycle 3 you will run Lighthouse on it: 100.

FOR POINTS AT ID. NAME IS WHAT GETS SENT.

for On the `label`. Holds the field's `id`, character for character. This is the whole reason clicking the words moves the cursor.

id Unique in the document. Copy-paste a field and forget to change it, and the click goes to the wrong box.

name The key the server receives. No `name`, no data — the field is ignored. Radios share one.

type `email` `tel` `number` `date` `radio` — each brings its own keyboard on a phone and its own validation, free. `required` stops an empty Send.

WHAT THE ADDRESS BAR SHOWS AFTER SEND (METHOD="GET")

```
<input type="email" name="email" id="a" required>
<input type="text" name="name" id="b">
<input type="date" id="c">
```

?

`email=maya%40ruppin.ac.il&name=Maya`

The third field has no `name`. It is on the page, the user filled it in, and it is not in that string.

PLACEHOLDER IS NOT A LABEL. A HINT IS NOT A NAME.

PLACEHOLDER ONLY

```
1 <input type="text" placeholder="Email" />
```

Disappears the moment you type. Cannot be clicked. Becomes the field's name only because nothing else did.

LABEL + FOR, HINT + ARIA-DESCRIBEDBY

```
1 <label for="email">Email</label>
2 <input type="email" id="email" name="email"
3     aria-describedby="email-hint" />
4 <p id="email-hint">Only for reminders.</p>
```

for points at id: clicking the label focuses the field. The hint is the field's *description*, wired with aria-describedby.

placeholder is not a label, always provide a real <label>.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

Ruppín Film Club

One film a week, in the small hall, for anyone on campus.

- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Alien on Tuesday 2 November at 19:30 in room 204. The Apartment on Tuesday 9 November at 19:30 in room 204. Spirited Away on Tuesday 16 November at 19:30 in the auditorium.

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Message

Compare — `live-code/01-form-that-fails-well/checkpoint.png` is this picture at 1280; `live-code/01-form-that-fails-well/end/` is the code behind it. Every label focuses its field, an empty Send stops on Full name, and a full Send writes five keys in the address bar.

CYCLE 2 · 25 MIN

THE TABLE THAT DESCRIBES ITSELF

Three films, three dates, three rooms, in one paragraph. It becomes a table — first the way everybody types it, with bold cells for headers, then the way a screen reader can walk it. Same pixels, different tree. Then the dates get a second spelling, for machines.

1 I DO

10 MIN

3 on the next two slides, 7 in the editor · live-code/02-table-describes-itself · caption · thead + th scope="col" · tbody + th scope="row" · time datetime

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 2 · columnHeader and rowheader in the tree

3 YOU DO

8 MIN

your own week-1 page: a real table with a caption and scoped headers, and a time on a date you already have · AI as tutor, checked in the tree

4 DEBRIEF

2 MIN

td strong is a cell · who reads datetime · ↗ reference

YOU DO · AI AS TUTOR

A TABLE IS A GRID OF RELATIONSHIPS

BOLD CELLS

```
<thead>
  <tr>
    <td><strong>Film</strong></td>
    <td><strong>Room</strong></td>
  </tr>
</thead>
```

Looks right. Validates. The Accessibility tree says cell "Film". A reader on 204 hears "204".

HEADERS WITH SCOPE

```
1 <caption>Screenings this month</caption>
2 <thead>
3   <tr>
4     <th scope="col">Film</th>
5     <th scope="col">Room</th>
6   </tr>
7 </thead>
8 <tbody>
9   <tr><th scope="row">Alien</th><td>204</td></tr>
10 </tbody>
```

columnheader "Film", rowheader "Alien". A reader on 204 hears "Room, 204" — and "Alien" before it. caption is the table's name.

TIME: THE TEXT IS FOR PEOPLE, DATETIME IS FOR MACHINES

ONE DATE, SPELLED TWICE

2027-11-02	a date
2027-11	a month
2019	a year
19:30	a time
2027-11-02T19:30	a date and a time

```
1 <td><time datetime="2027-11-02T19:30">Tuesday 2 November,
```

A wrong format — 2 Nov 2027 — is not an error anywhere. Not in the browser, not in the validator. The machine simply gets a string it cannot read.

The text: anything a person reads. The `datetime`: one format, `YYYY-MM-DD`, then `T` and the time. Google, a calendar, and week 11's JavaScript read that one — none of them reads "Tuesday".

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS

Ruppín Film Club

One film a week, in the small hall, for anyone on campus.

- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Film	Date	Room
Alien	Tuesday 2 November, 19:30	204
The Apartment	Tuesday 9 November, 19:30	204
Spirited Away	Tuesday 16 November, 19:30	Auditorium

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Compare — `live-code/02-table-describes-itself/checkpoint.png` is this picture at 1280; `live-code/02-table-describes-itself/end/` is the code behind it. In the Accessibility tree: `table "Screenings this month"` with three `columnheader` and three `rowheader`; `$$('th[scope]').length` is 6.

CYCLE 3 · 30 MIN

THE READER WHO CANNOT SEE

Same page, no mouse, no eyes. The first accessibility report, its red rows read aloud and fixed; a skip link and a main that can take focus; a nav that says where you are; and one "fix" for the Tab order that breaks everything it touches.

1 I DO

10 MIN

2 on the next slide, 8 in the browser
· live-code/03-reader-who-cannot-see · Lighthouse · h4 to h3 · skip link + main id + tabindex="-1" · tabindex="1" and why not · aria-current="page"

2 WE DO

8 MIN

you type it on your copy,
Lighthouse before and after ·
compare with checkpoint 3 · first
Tab is Skip to content

3 YOU DO

10 MIN

your own week-1 site, both pages:
skip link, main, aria-current,
Lighthouse, every red row read

4 DEBRIEF

2 MIN

what the report saw, what it cannot
· ↗ reference

YOU DO · AI-FREE

PRESS TAB. WHAT IS THE FIRST THING YOU REACH?

THE WAY IN

Skip link

```
1 <body>
2 <a href="#main">Skip to content</a>
3 <header>...</header>
4 <nav>...<a href="index.html" aria-current="page">Home</a>...</nav>
5 <main id="main" tabindex="-1">...</main>
6 </body>
```

tabindex="-1"

The first thing Tab reaches; it jumps past the header and the nav to `main`.

Lets `main` take focus.

Without it `activeElement` stays `body`. Never a positive number — that means "before everything".

Headings

On the line, nav id for the `h2` then the `h4` is the first `h4` in the page; next week it is the selector.

THE FIRST REPORT. GREEN IS A FLOOR.

DevTools · Lighthouse · Accessibility · Analyze. One row per axe rule id (measured, Lighthouse 12.8):

- heading-order** "Heading elements are not in a sequentially-descending order" — the `h4` . Score 93.
- tabindex** "Some elements have a [tabindex] value greater than 0" — serious.
- label · image-alt** A field with no name at all; an `img` with no `alt` — critical.
- target-size** WCAG 2.2, needs CSS. Stays red; the grader (`wcag2a` , `wcag2aa`) does not count it.

WHAT IT CANNOT SEE

The placeholder-only form: **100.** `aria-label` under words you cannot click: **100.** Bold `td` s as headers: nothing.

It checks that a name exists. Not that the name is the words on the screen, or that clicking them works. That is you: click, Tab, the tree.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

[Skip to content](#)

Ruppín Film Club

One film a week, in the small hall, for anyone on campus.

- [Home](#)
- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Film	Screenings this month		Room
	Date		
Allen	Tuesday 2 November, 19:30	204	
The Apartment	Tuesday 9 November, 19:30	204	
Spirited Away	Tuesday 16 November, 19:30	Auditorium	

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Compare — `live-code/03-reader-who-cannot-see/checkpoint.png` is this picture at 1280; `live-code/03-reader-who-cannot-see/end/` is the code behind it. First Tab: Skip to content. Enter, then Tab: the Full name field. Lighthouse: one red row, `target-size`, and everybody knows why.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and DevTools on Lighthouse.

CYCLE 4 · 25 MIN

"REVIEW MY FORM FOR ACCESSIBILITY — IS ARIA-LABEL ENOUGH?"

A student who ran Lighthouse, read one thing about `aria-label`, and asks the chat to confirm it. What comes back is confident, correct about the tool it quotes, and answers the question exactly as asked. Then the room clicks the words.

1 I DO

7 MIN

2 on the next slide, 5 in the chat ·
ai-lab/04-review-my-form · P1 live,
then RUN the answer: Lighthouse
100, click the words, the tree ·
transcript.md if the room got
something else

2 WE DO

6 MIN

everyone pastes P1 · run what
came back · find what the tool did
not check, before it is named

3 YOU DO

10 MIN

write P2 as a checkable constraint ·
run it · one question about your
own form · the journal line

4 DEBRIEF

2 MIN

a leading question gets the answer
it asked for · ↗ the journal
exemplar

CYCLE 4 · THE GAP

AXE PASSES. LIGHTHOUSE SAYS 100. NOW CLICK THE WORDS.

WHAT THE TOOL CHECKS

That every field has an accessible **name**. `aria-label="Full name"` is a name; so is `aria-labelledby` pointing at the words. So is a placeholder, when nothing else is there. Rule `label`: green.

WHAT IT DID NOT CHECK

That the name is the words on the screen. That clicking them focuses the field (it does not — `activeElement` stays `body`). That the hint survives typing (it does not). Three measurements, none of them in the report.

THE CONSTRAINT, NOT THE QUESTION

"Is aria-label enough?" is a question the answer can agree with. "Every field gets a visible `label` for ; clicking the words must focus the field" is a constraint you can check. Your journal line: *claim · how I checked · verdict.*

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you **run** the answer to P1 — Live Server, Lighthouse, then a click on the words — instead of stopping at the 100?
2. Does your P2 state a constraint the browser can check — "clicking the words focuses the field", "label for matches the id" — and does what came back meet it, on all four words?
3. Does your journal line name **a tool and a measurement** — "Lighthouse 100", "click: nothing", "Name from aria-label" — and not just "Claude was right / wrong"?

Compare — `ai-lab/04-review-my-form/checkpoint-he.md` has the same three questions and the measurements; `artifact/claude-p1/` and `artifact/claude-p2/` are the two forms, ready to open side by side.

CYCLE 5 · 20 MIN

TWO THINGS THAT MOVE, ZERO LINES OF JAVASCRIPT

A section that opens and closes, and a banner that picks its own source when the window shrinks. Both are HTML. Both work with the keyboard, in a screen reader, and on the day your JavaScript throws.

1 I DO

7 MIN

2 on the next slide, 5 in the editor ·
live-code/05-behaviour-without-javascript · details + summary ·
picture: source first, img last

2 WE DO

5 MIN

you type it on your copy · compare
with checkpoint 5 · WIDE at 1280,
NARROW at 375, no scrollbar

3 YOU DO

6 MIN

your own page: one details, one
picture with two sources · AI as
tutor, checked at two widths and in
the validator

4 DEBRIEF

2 MIN

which element the reader meets ·
what happens when the JS breaks ·
↗ reference

YOU DO · AI AS TUTOR

DETAILS OPENS ITSELF. PICTURE CHOOSES ITSELF. IMG IS LAST.

OPENS AND CLOSES, SPACE OR CLICK

```
1 <details>
2   <summary>How screenings work</summary>
3   <ul>...</ul>
4 </details>
```

`summary` is what you always see, and it is in the Tab order by itself. Open it and the browser writes `open` on the `details`; write it yourself to start open.

SOURCE FIRST, IMG LAST

```
1 <picture>
2   <source media="(min-width: 700px)" srcset="images/banner-wide."
3   
```

The browser walks the `source` s in order and stops at the `img` `source` after it is never read — the narrow banner at every the validator says `element-permitted-order`. The `alt`, `height` live on the `img`: it is the element the reader meets

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS

[Skip to content](#)

Ruppin Film Club

One film a week, in the small hall, for anyone on campus.

Ruppin Film Club

WIDE 640x160 - shown from 700px



- [Home](#)
- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Film	Date	Room
Alien	Tuesday 2 November, 19:30	204
The Apartment	Tuesday 9 November, 19:30	204
Spirited Away	Tuesday 16 November, 19:30	Auditorium

► How screenings work

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Compare — `live-code/05-behaviour-without-javascript/checkpoint.png` is this picture at 1280; `live-code/05-behaviour-without-javascript/end/` is the code behind it. Device toolbar at 375: the NARROW banner and no scrollbar. Space on the summary: the list opens.

ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Your week-1 page, copied into week-02/ : the contact form with `fieldset`, `label` for, `name`, two input types and `required` (cycle 1) · a table with `caption` and `th scope`, a `time datetime` (cycle 2) · skip link, `main id tabindex="-1"`, `aria-current="page"`, headings in order, Lighthouse read, validator clean, no `CODE HERE` left (cycle 3). Three commits.

EXTEND · STRETCH AND CHALLENGE

A hint with `aria-describedby`, a `details`, a `picture` with the `img` last, `th scope="row"`, and `about.html` with the same three lines (cycles 1, 2, 3, 5). Then: the picture fits at 375, a radio group, and no `aria-label` anywhere — the words on the screen are the label (cycle 4).

THE RULES, AND THE EXAM

No CSS. No JavaScript. No `aria-label` on fields. No positive `tabindex`. AI as tutor — it reviews, you type, you click. Submit the repo URL and the SHA in Moodle. Week 13 asks you about this code, offline.

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

1. A form validates clean and Lighthouse says 100. Name two things that can still be wrong with it, and the test for each.
2. A screen reader lands on the cell `204`. What does it say in a table of bold `td`s, and what does it say once the headers are `th` scope?
3. You press Tab on a fresh page. What should you reach first, what attribute lets `main` take focus, and which attribute value would break the whole order?

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: every input type, get or post, every datetime format, how a field's accessible name is computed, ARIA's five rules, the WCAG 2.2 rows this week touches, and the axe rule ids you saw in the report. The next slide lists it.

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · Forms, tables, and the reader who cannot see — the catalogue](#) ← spine

26

[2 · Forms — the input types worth knowing](#) ← spine 5

[3 · The form, done properly](#) ← spine 5

[4 · id, name, for — three attributes, three jobs](#) ← spine 5

[5 · required, and the browser's own validation](#) ← spine 5

[6 · get or post](#) ← spine 5

[7 · Tables — the full anatomy](#) ← spine 10

[8 · datetime — every valid format](#) ← spine 10

[9 · details and summary — the attributes](#) ← spine 22

[10 · picture vs srcset and sizes](#) ← spine 22

[11 · The accessibility tree, and where a name comes from](#) ← spine 19

[12 · ARIA's five rules](#) ← spine 19

[13 · WCAG 2.2 — the rows this week touches](#) ← spine 16

[14 · The axe rules you saw this week](#) ← spine 16

[15 · The skip link, line by line](#) ← spine 14

[16 · aria-current — the values](#) ← spine 14

[17 · Headings are an outline](#) ← spine 14

[18 · Where the tree lives in DevTools](#) ← spine 14

[19 · The validator, axe, and a click — who sees what](#) ← spine 19